Grow

Guía de auditoría, decisiones y plan de trabajo

Todo lo analizado y decidido en la sesión de auditoría funcional y de producto de octubre de 2026: hallazgos, decisiones con su porqué y su valor comercial, explicaciones técnicas para estudiar, orientación legal y el paso a paso para terminar la aplicación antes de ofrecerla.

Versión 1 · 23 de octubre de 2026
Proyecto: plataforma modular de pedidos para growshops físicos (retiro en el local)
Documentos relacionados en el repositorio: `docs/arquitectura/arquitectura-v13.md`, `docs/auditoria/auditoria-funcional-2026-10.md`, `docs/auditoria/roadmap-funcionalidades.md`, `docs/auditoria/pasarela-de-pago.md`
Lo legal de este documento es orientación general, no asesoramiento legal.

Índice

1. Cómo usar este documento
2. Qué es Grow y cómo está construido
3. Cómo se hizo la auditoría
4. Hallazgos de la auditoría
5. Decisiones tomadas (con porqué y valor comercial)
6. Explicación técnica: el mapa general
7. Explicación técnica: hosting, CDN y builds
8. Explicación técnica: DNS y dominios
9. Explicación técnica: cuentas y titularidad durante las cuotas
10. Explicación técnica: Supabase free, tráfico e imágenes
11. Explicación técnica: backups
12. Explicación técnica: cifrado con age y la "doble llave"
13. Explicación técnica: el flujo de reserva y los avisos
14. Explicación técnica: pasarela de pago (para el futuro)
15. Orientación legal
16. Plan de trabajo paso a paso
17. Preguntas abiertas
18. Glosario
19. Fuentes consultadas

1. Cómo usar este documento

Este documento tiene dos lectores:

- **Vos**, para entender qué se analizó, qué se decidió y por qué, y para estudiar los conceptos técnicos que aparecieron (DNS, CDN, caché, backups, cifrado, webhooks). Las secciones 6 a 14 están escritas como material de estudio, con diagramas.
- **Una futura sesión de Claude Code** que retome el trabajo de código. Para ella son clave la sección 5 (decisiones, que no hay que volver a discutir), la 16 (plan paso a paso con archivos y criterios de terminado) y la 17 (lo que todavía falta decidir).

> **Para una futura sesión de Claude Code**
>
> Leé primero `CLAUDE.md` y `docs/arquitectura/arquitectura-v13.md`. Después las secciones 5, 16 y 17 de este documento. Las decisiones de la sección 5 ya están tomadas por el dueño del proyecto; si una tarea parece contradecirlas, preguntá antes de cambiar el rumbo. Respetá las reglas del proyecto: cambios de base solo incrementales, regenerar tipos con cada columna nueva, client y admin aislados, nada de datos de un negocio en el código.

Convenciones: los archivos se citan como `ruta:línea` según el código del 23/10/2026; las líneas pueden moverse con cambios posteriores. La severidad de los hallazgos es **alta** (bloquea vender o expone a un problema real), **media** (afecta la operación o la conversión) o **baja** (mejora o deuda).

2. Qué es Grow y cómo está construido

Grow es una plataforma de pedidos para growshops físicos que no tienen e-commerce. Se vende de forma íntegra a cada negocio (no es una suscripción tipo SaaS). El cliente final arma un pedido en la tienda pública y lo retira en el local: no hay envíos ni cobro online. El dueño gestiona pedidos, catálogo y configuración desde un panel privado (el "admin").

2.1 Las piezas

Pieza	Qué es	Dónde vive
Client (tienda pública)	Aplicación React que ven los compradores: catálogo, ficha, carrito, reserva, cuenta.	`client/`
Admin (backoffice)	Aplicación React separada para el dueño: pedidos, productos, categorías, paneles, clientes, configuración.	`admin/`
Shared	Tipos, cliente de Supabase, conversores y utilidades comunes, sin React.	`shared/` (incluye los SQL)
Supabase	Backend como servicio: base Postgres, login, permisos por fila (RLS), funciones en la base (RPC), almacenamiento de imágenes, tareas programadas y funciones en la nube (Edge Functions).	Una instancia por negocio; código en `shared/*.sql` y `supabase/functions/`
Resend	Servicio para enviar emails.	Cuenta por negocio
Hosting	Donde se publican client y admin. Hoy Vercel; decidido pasar a Cloudflare Pages (sección 5).	Dos proyectos, uno por app

2.2 Principios que no se negocian

- **Genericidad:** ningún dato de un negocio en el código. Todo vive en la base y se edita desde el admin. Excepciones deliberadas: colores del tema, logo y las reseñas de ejemplo.
- **Una instancia por negocio:** cada growshop tiene su propia base de Supabase, sin mezclar datos.
- **Client y admin aislados:** dos apps independientes; si algo hace falta en ambas, se duplica.
- **La seguridad real está en la base** (RLS y funciones): la anon key es pública, así que el navegador nunca es fuente de verdad (por ejemplo, los precios del pedido los calcula la base).

3. Cómo se hizo la auditoría

Se pidió una auditoría funcional y de producto, sin modificar archivos del proyecto, con cinco tareas: comparar el documento de arquitectura con el código; evaluar cada funcionalidad (si funciona, si aporta valor, casos borde); proponer funcionalidades faltantes; analizar riesgos (reseñas, sellos, normativa de venta a distancia, rubro, datos personales); y evaluar una pasarela de pago.

3.1 Qué se revisó

- Todo `client/`, `admin/`, `shared/`, `supabase/`, los `vercel.json` y el workflow de GitHub.
- `npm run typecheck` (pasa en los tres paquetes) y los builds (compilan; el admin avisa un archivo de 1,27 MB porque la librería de Excel no se carga bajo demanda).
- La tienda corriendo en local contra la base real, para medir el peso real de las imágenes (sección 10).
- Normativa y documentación oficial (Boletín Oficial, argentina.gob.ar, InfoLEG, AAIP, DNDA, Mercado Pago, Supabase, Cloudflare, Vercel, Meta), con fecha de consulta 23/10/2026.

3.2 Límites

- No hubo acceso al panel de Supabase, Vercel ni Resend: lo que depende de su configuración real quedó marcado como no verificable.
- El `.docx` de arquitectura no está en el repositorio; se usó el `.md`, que es su fuente.
- Lo legal es orientación, no asesoramiento.

4. Hallazgos de la auditoría

El detalle completo, con evidencia y recomendación por hallazgo, está en `docs/auditoria/auditoria-funcional-2026-10.md`. Acá va lo esencial, actualizado con lo que se descubrió después en la conversación.

4.1 Los cinco más importantes

#	Hallazgo	Severidad	Qué se decidió
1	Las reseñas fijas eran de un growshop real (nombres, fotos, detalles del local), con un promedio y un total inventados.	**Alta**	Reseñas y avatares genéricos, sin referencias a ninguna tienda (5.1).
2	El dueño no se entera de un pedido nuevo salvo que abra el admin (el aviso quedó como `TODO` en `ResumePage.tsx:151`).	**Alta**	WhatsApp desde el cliente + email automático al dueño (5.3).
3	Editar un producto borra y recrea todas sus variantes (`ProductForm.tsx:121`): el historial pierde qué variante se vendió, los carritos abiertos fallan y, si algo se corta, el producto queda sin variantes.	**Alta**	Corregir (tarea 1.4 del plan).
4	Faltan textos legales coherentes con el modelo (Términos, privacidad) y la interfaz habla de "compra".	**Alta**	Modelo de "reserva", Términos nuevos y política de privacidad (5.8, 5.9).
5	No hay backups: el plan free de Supabase no los incluye.	**Alta**	Backup cifrado cada 3 días en dos destinos (5.6).

4.2 Descubierto después del informe

- **La compresión de imágenes no existe en el código.** Existe la constante `IMAGEN_PRODUCTO_PESO_OBJETIVO_KB` (30–50 KB) pero nadie la usa, y su comentario menciona una "Edge Function de resize" que no existe. El admin sube el archivo tal cual (`ProductForm.tsx:97-99`); el único freno es el límite de 5 MB del bucket. Las imágenes actuales son livianas porque se comprimieron a mano antes de subirlas.
- **Las fotos del catálogo no tienen carga diferida** (`loading="lazy"`) y se sirven con caché de una hora (`Cache-Control: max-age=3600`).
- **El teléfono de la tienda de prueba** está cargado como `1549982364`; el botón de WhatsApp arma `wa.me/1549982364`, que WhatsApp interpreta con el código de país 1 (Estados Unidos). Confirma la necesidad de normalizar teléfonos.
- **El plan Hobby de Vercel no permite uso comercial**, que es el caso de una tienda (sección 7).

4.3 Resto de los hallazgos, en una tabla

Código	Hallazgo	Sev.
C1	Carrito solo en memoria: se pierde al recargar; "Ver Producto" abre otra pestaña con otro carrito.	**Media**
C2	El carrito guarda el precio al agregar; la base cobra el del catálogo al confirmar. Pueden diferir y la pantalla final no muestra el total real.	**Media**
C3	Si un producto se agota entre el carrito y la confirmación, la base lo rechaza con mensaje, pero el carrito no marca cuál.	**Baja**

C4	Sin clave de idempotencia: un reintento por corte de red puede duplicar el pedido.	**Baja**
C5	La confirmación no muestra número de pedido, local ni horario.	**Media**
C6	No hay "olvidé mi contraseña"; el login muestra errores en inglés.	**Media**
C7	Las variantes no pueden estar "sin stock"; si todas están pausadas el botón no hace nada.	**Baja**
C9	Horario único (apertura/cierre) sin días ni horarios por sucursal.	**Media**
C10 / P5	Teléfonos sin normalizar: la tienda arma `wa.me` sin código de país; el admin antepone `54` sin el `9` de los celulares argentinos.	**Media**
P2	Cancelar no guarda motivo ni avisa al cliente.	**Media**
P3	Pedidos "listos para retirar" que nadie busca quedan abiertos para siempre.	**Media**
P4	La búsqueda de pedidos es solo por nombre.	**Baja**
P6	La API devuelve máximo 1000 filas: con volumen, Pedidos y Clientes quedarían truncados sin aviso.	**Media**
A2	Al crear un producto, su id se busca por nombre; con nombres repetidos falla a mitad de camino.	**Media**
A3	El precio base acepta 0.	**Baja**
A4	Imágenes reemplazadas o de productos borrados quedan huérfanas en el bucket.	**Baja**
A5	Logo de 720 KB; bundle del admin de 1,27 MB.	**Baja**
D5 / D6	El CHECK de `clientes` y los límites del bucket solo existen en producción (vía `correcciones.sql`), no en `shared/*.sql`: una instancia nueva nacería sin ellos.	**Media**
D7	`pedido_items` no guarda el nombre de la variante; se lee en vivo y se pierde si la variante cambia.	**Alta**
D10	El keep-alive consulta la raíz de la API, no una tabla; no está claro que cuente como actividad.	**Baja**

4.4 Lo que sobra o está sobrediseñado

- **Paneles de navegación** (3 tablas, cards con imágenes, orden fijo o al azar): mucho para un growshop con 5–15 categorías. No borrar, pero no invertir más.
- **"Compra Protegida"** en tres pantallas: no tenía respaldo. Se reemplaza (5.2).
- **"Compartir en Instagram"** solo copia el link: el rótulo promete otra cosa.
- **Cron cada minuto** para limpiar el mensaje de alerta: la tienda ya lo oculta al vencer.

5. Decisiones tomadas

Cada decisión tiene su porqué y, cuando aplica, el valor comercial que aporta al venderle Grow a un growshop. Estas decisiones las tomó el dueño del proyecto durante la sesión: no se vuelven a discutir salvo que aparezca un hecho nuevo.

5.1 Reseñas de ejemplo genéricas

Decisión: reemplazar las 10 reseñas actuales por reseñas inventadas y genéricas (nombres ficticios, textos aleatorios sin referencias a ninguna tienda, horario, empleado ni medio de pago) y avatares ilustrados generados (SVG), variados, que no se basen en personas reales. Se elimina el resumen "4,8 (232)". Mientras la aplicación es una demo que solo usa el desarrollador, no lleva aviso de "reseñas ficticias"; cuando un cliente contrata, en la etapa de personalización se reemplazan por reseñas reales del negocio o se oculta la sección.

Por qué: las reseñas originales eran de un growshop real que además está en la lista de clientes a contactar. Mostrarlas en otra tienda era engañoso, usaba nombre e imagen de terceros sin consentimiento y rompía la genericidad. Un avatar "parecido" a la persona real habría reforzado el problema.

Valor comercial: la demo luce completa sin exponer a nadie, y al vender se ofrece "reseñas reales de tus clientes" como parte de la personalización.

5.2 "Datos protegidos" en lugar de "Compra Protegida"

Decisión: quitar el sello "Compra Protegida" del catálogo y la ficha. En el checkout (reserva), mostrar un sello "Datos protegidos".

Por qué: no hay pago online ni programa de protección que respalde "Compra Protegida", y es además el nombre de un programa de Mercado Pago. "Datos protegidos" sí es verdad: HTTPS, base cifrada en reposo, permisos por fila y backups cifrados. Necesita la política de privacidad detrás (5.9) para que la promesa tenga respaldo.

Texto sugerido: "Datos protegidos — Usamos tus datos solo para gestionar tu reserva" con link a la política de privacidad.

5.3 Aviso de pedido nuevo: WhatsApp del cliente + email al dueño

Decisión: al confirmar la reserva, la base crea el pedido y se abre el WhatsApp del comprador (app en el celular o WhatsApp Web en la computadora) con un mensaje ya escrito al número del local: número de reserva, productos, cantidades, total y nombre. En paralelo, y siempre, sale un email automático al dueño a una casilla propia configurable en el admin (campo nuevo en Configuración).

Por qué: un sistema no puede mandar un WhatsApp sin un número que envíe. Las opciones eran: que lo mande el propio comprador (gratis, sin cuentas de Meta, abre la conversación donde igual se cierra el trato); la API oficial de WhatsApp con un número "maestro" (automática pero con costo por mensaje, verificación del negocio en Meta y plantillas aprobadas; y si el número es del desarrollador, ese costo y dependencia sobreviven a la entrega); o librerías no oficiales (violan los términos de Meta y arriesgan el bloqueo del número). Se eligió la primera. El email cubre el caso en que el comprador no envía el WhatsApp (por ejemplo, desde una computadora sin WhatsApp Web).

Valor comercial: "te llegan las reservas al WhatsApp del local y al email, sin costo mensual".

5.4 Sin cobro online por ahora

Decisión: la primera versión es de pedidos online con retiro y pago en el local. La pasarela queda documentada (sección 14 y `pasarela-de-pago.md`) como mejora futura según cada cliente.

Por qué: simplifica lo legal (el contrato se perfecciona en el local), evita comisiones y evita el riesgo de que el procesador suspenda la cuenta por el rubro (los términos de Mercado Pago, según notas de prensa, prohíben la parafernalia).

5.5 Infraestructura: Supabase free y Cloudflare Pages

Decisión: cada negocio arranca con Supabase free y pasa a Pro cuando el tráfico lo justifique. El hosting de client y admin pasa de Vercel a **Cloudflare Pages** (plan gratuito).

Por qué: el plan Hobby de Vercel prohíbe el uso comercial y no hay plan intermedio (Pro cuesta USD 20 por mes por asiento). Cloudflare Pages es gratuito y, según su personal, permite uso comercial; sin tope publicado de tráfico estático; y tiene una función clave para el modelo de cuentas: puede servir un dominio que está en otra cuenta de Cloudflare (sección 9). Cloudflare recomienda Workers para proyectos nuevos, pero esa función es exclusiva de Pages.

Valor comercial: costo mensual de infraestructura $0 al inicio; el cliente solo paga el dominio (unos $8.500 por año en NIC.ar al momento de la conversación).

5.6 Backups cada 3 días, cifrados, en dos destinos

Decisión: un proceso automático (GitHub Actions) exporta la base y las imágenes cada 3 días, las comprime, las cifra con **age** para dos destinatarios (el dueño y el desarrollador) y las sube a Cloudflare R2 (destino principal) y al Google Drive del dueño (copia visible). Retención aproximada de 2–3 meses (a definir; propuesta en 11.5). Avisos de falla al desarrollador y al dueño, más un resumen mensual.

Por qué: Supabase free no hace backups; los datos son del dueño desde el primer día; dos destinos distintos cumplen la regla 3-2-1.

Valor comercial: "tus datos tienen copia de seguridad cifrada cada 3 días, en dos lugares, con una llave que tenés vos, y te llega un resumen todos los meses".

5.7 Imágenes: todas las técnicas de optimización

Decisión: implementar todas las técnicas de la sección 10.4: compresión automática en el admin, dos tamaños (miniatura y grande), caché larga, carga diferida, paginación, columnas justas, no repetir consultas, borrar huérfanas, logo liviano y monitoreo. Tamaños decididos: **miniatura de 400 px** de lado mayor (objetivo 8–15 KB) y **grande de 1000 px** (objetivo 30–60 KB), ambas en WebP.

Por qué: el límite que puede romper el plan free no es la cantidad de visitas sino el peso de las imágenes (sección 10).

5.8 Modelo de "reserva" y textos legales

Decisión: la tienda y el admin hablan de **reservas**, no de compras. Los Términos dirán que la reserva es una solicitud sin obligación de pago y que el contrato de compraventa se perfecciona en el local con el pago y la entrega. La oferta indicará sus limitaciones (sujeta a stock; precios con vigencia) y el tratamiento del error de precio. No se implementa botón de arrepentimiento en esta etapa (análisis en 15.1). Los datos del proveedor (razón social, CUIT, domicilio) no se cargan hasta que haya un cliente real; los campos quedan preparados vacíos.

Por qué: la postura "es una reserva, no una venta a distancia" es defendible solo si todo el sistema es coherente con ella: interfaz, mensajes, emails y Términos.

5.9 Política de privacidad

Decisión: dejar preparada una política de privacidad genérica (Ley 25.326) que se completa con los datos del responsable al personalizar.

5.10 Teléfonos normalizados

Decisión: normalizar todos los teléfonos (comprador, cliente registrado y teléfono del local) a formato internacional argentino (`549` + código de área + número, sin 0 ni 15) y validarlos al cargarlos. Queda abierto si se piden en uno o dos campos (sección 17).

5.11 Licencia no exclusiva y DNDA

Decisión: al completar las tres cuotas, el cliente recibe una **licencia perpetua y no exclusiva** con entrega del código fuente de su instancia, en lugar de la "titularidad" del código. El desarrollador conserva la autoría del código base. Evaluar el depósito del código base en la DNDA como obra inédita.

Por qué: el mismo código base se licencia a varios growshops. Ceder la titularidad a uno impediría seguir licenciándolo, y reescribir el código para "esquivar" esa cesión no salva el problema, porque una versión modificada sigue siendo obra derivada. La licencia resuelve el problema de raíz.

5.12 Modelo comercial y de cuentas (contexto para todo lo anterior)

- Precio en tres cuotas; al completar el pago se entrega el código (licencia), la cuenta de hosting y el control total.
- El dueño compra su dominio `.com.ar` a su nombre en NIC.ar y crea su cuenta de Cloudflare (DNS y R2) y su cuenta de Supabase desde el primer día; invita al desarrollador como miembro.
- Durante las cuotas, los proyectos de hosting viven en la cuenta del desarrollador; en caso de mora, se dan de baja. Los datos siguen siendo del dueño.
- El contrato lo redacta el desarrollador (estudiante de abogacía, cuarto año), en una carilla, con obligaciones de ambas partes, servicio técnico y consecuencias de la mora.

6. Explicación técnica: el mapa general

Antes de los detalles, el dibujo completo de cómo quedará funcionando un growshop con las decisiones tomadas.

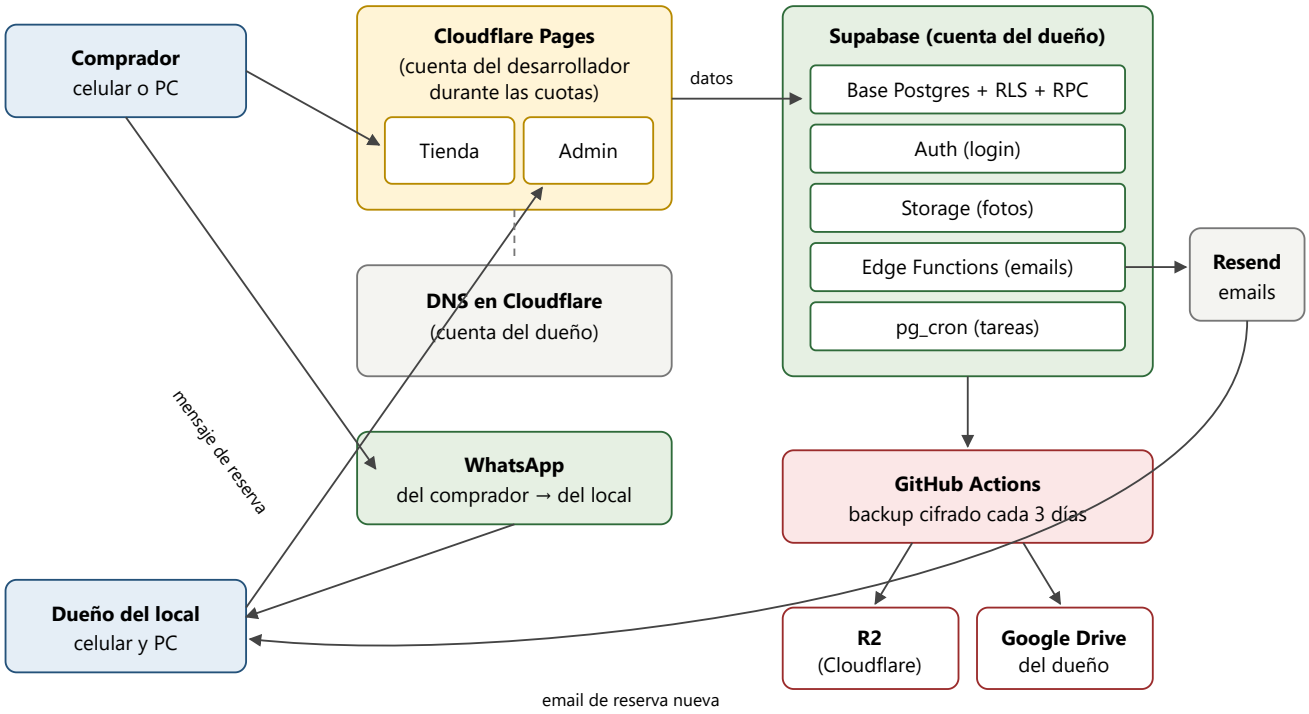


Figura 1. Mapa general con las decisiones tomadas. Las líneas punteadas indican que el DNS "apunta" el dominio hacia el hosting.

Lectura del dibujo: el comprador abre la tienda (servida por Cloudflare Pages); la tienda le pide datos a Supabase; al reservar, se abre su WhatsApp con el mensaje al local y Supabase dispara un email al dueño vía Resend. El dueño gestiona desde el admin, que también habla con Supabase. Cada 3 días, GitHub Actions copia la base y las fotos, las cifra y las deja en R2 y en Google Drive.

7. Explicación técnica: hosting, CDN y builds

7.1 Qué hace un hosting de sitios estáticos

Grow, al compilarse (`npm run build`), se convierte en un puñado de archivos fijos: un `index.html`, archivos JavaScript y CSS, y algunas imágenes (logo, avatares). No hay un servidor propio que "piense": toda la lógica de datos la resuelve Supabase. Por eso alcanza con un hosting de **sitios estáticos**, cuyo trabajo es guardar esos archivos y entregarlos rápido.

7.2 Qué es una CDN

Una CDN (red de distribución de contenido) es una red de servidores repartidos por el mundo que guardan copias de los archivos. Cuando alguien en Córdoba abre la tienda, los archivos le llegan desde el servidor más cercano, no desde un único servidor lejano. Resultado: más velocidad y más resistencia a picos de tráfico. Cloudflare tiene una de las redes más grandes del mundo, con presencia en Argentina.

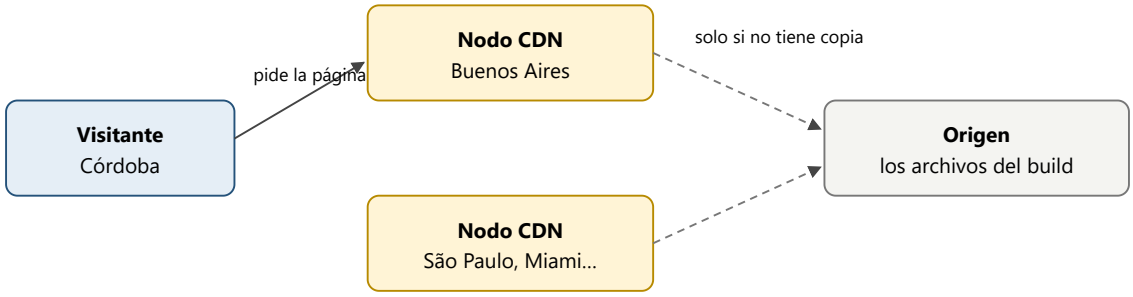


Figura 2. La CDN responde desde el nodo más cercano; solo va al origen la primera vez o cuando cambian los archivos.

7.3 "Sin tope publicado de tráfico estático": qué significa

Cloudflare promociona el tráfico de Pages como ilimitado y su página de límites no fija un máximo de ancho de banda ni de visitas para archivos estáticos. Sí rige el uso razonable de sus términos de servicio (no se puede, por ejemplo, usar Pages para distribuir archivos de video masivos). Para una tienda normal, el tráfico de Pages no es un límite práctico.

Cuentas con números reales del build actual: por visita, la tienda descarga unos 160 KB comprimidos de JavaScript y CSS, más el logo (720 KB hoy, unos 30 KB una vez optimizado), más las fuentes de Google. Con 500 visitas diarias:

Concepto	Por visita	500 visitas/día	Por mes
JS + CSS (comprimidos)	~160 KB	~80 MB	~2,4 GB
Logo actual / optimizado	720 KB / 30 KB	360 MB / 15 MB	10,8 GB / 0,45 GB

Para Cloudflare esos volúmenes son insignificantes, y además el navegador guarda en caché los archivos y la mayoría de las visitas repetidas no descargan nada. **Con 500 visitas diarias el cuello de botella no es el hosting sino el egress de Supabase** (sección 10): ahí sí conviene pasar a Supabase Pro (unos USD 25 por mes), que con ese volumen de ventas se paga sola, tal como planteaste.

> **Matiz sobre el uso comercial**
>
> Que el plan gratuito de Pages permite uso comercial lo afirma el personal de Cloudflare en su foro y sus condiciones no lo prohíben, pero no encontré una frase explícita en la documentación oficial. Es un riesgo bajo; si algún día cambiara, el plan pago de Workers cuesta unos USD 5 por mes.

7.4 Qué es un build y qué significan "500 builds por mes"

Un **build** (compilación) es el proceso que transforma el código fuente (TypeScript, React, CSS Modules) en los archivos finales que entiende el navegador. Es lo que hace `npm run build`. Cloudflare Pages se conecta al repositorio de GitHub y, **cada vez que se sube un cambio a la rama principal** (un *push*), hace el build en sus servidores y publica el resultado. Cada una de esas compilaciones cuenta como un build.

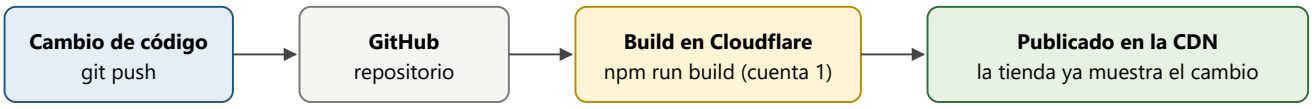


Figura 3. Cada push a la rama principal dispara un build.

¿500 por mes es poco o mucho? Depende de la etapa:

- **Tienda funcionando, solo mantenimiento:** entre 2 y 20 builds por mes. Muy por debajo del límite. Ojo: cargar productos, precios o fotos **no** requiere build, porque eso vive en Supabase.
- **Desarrollo intenso** (como la etapa que viene): pueden ser 10 a 30 por día si cada cambio chico se sube por separado. Ahí sí puede acercarse al límite. Solución: desarrollar y probar en local, y subir en tandas.
- **Dos proyectos por cliente:** la tienda y el admin están en el mismo repositorio. Sin configuración, un cambio que solo toca el admin podría disparar el build de los dos. Pages permite definir rutas vigiladas (*build watch paths*) para que cada proyecto se compile solo si cambió su carpeta (o `shared/`).
- **Varios clientes en tu cuenta:** los builds concurrentes se cuentan por cuenta y la documentación no aclara si los 500 mensuales son por cuenta o por proyecto. Con varios clientes en mantenimiento liviano sigue sobrando, pero conviene saberlo.

Otros límites del plan gratuito de Pages: 1 build a la vez, 20 minutos por build (el de Grow tarda segundos), 20.000 archivos por sitio, 25 MB por archivo, 100 dominios por proyecto y 100 proyectos por cuenta.

7.5 Qué cambia al pasar de Vercel a Pages

En Vercel (hoy)	En Cloudflare Pages
`vercel.json`: rewrite de todas las rutas a `index.html`	Archivo `_redirects` (o el modo SPA automático de Pages)
`vercel.json`: headers de seguridad (CSP, nosniff, etc.)	Archivo `_headers` en `public/`, mismo contenido
Variables `VITE_SUPABASE_*` en el panel	Igual, en el panel de Pages; cambiarlas requiere un nuevo build
Root Directory `client/` o `admin/`	Igual: directorio raíz y comando de build por proyecto

8. Explicación técnica: DNS y dominios

8.1 Qué es el DNS

El DNS (sistema de nombres de dominio) es la **guía telefónica de Internet**. Las personas recuerdan nombres (`canopia.com.ar`), pero las computadoras se conectan a direcciones numéricas. Cuando alguien escribe el nombre, su dispositivo pregunta "¿dónde está esto?" y el DNS responde.

Esa guía está hecha de **registros**. Los que vamos a usar:

Tipo	Para qué sirve	Ejemplo en Grow
CNAME	"Este nombre es un alias de este otro nombre."	`admin.canopia.com.ar` → el proyecto del admin en Pages
A / AAAA	"Este nombre está en esta dirección numérica."	Lo maneja Cloudflare automáticamente para el dominio principal
TXT	Texto libre, usado para demostrar que el dominio es tuyo.	Verificación del dominio en Resend (para mandar emails en nombre de la tienda)
MX	"El email de este dominio se recibe en tal servidor."	Si el dueño quiere casillas `@canopia.com.ar` (Resend usa uno para rebotes)

8.2 Los tres actores

1. **El registro del dominio (NIC Argentina):** donde se compra y renueva el `.com.ar`, a nombre del titular (CUIT/CUIL del dueño). NIC.ar no contesta la guía telefónica: solo dice **quién** la contesta (eso se llama "delegación").
2. **El proveedor de DNS:** el servidor que contesta la guía. Cloudflare lo ofrece gratis, con buena seguridad.
3. **El hosting:** donde están los archivos de la web (Pages).

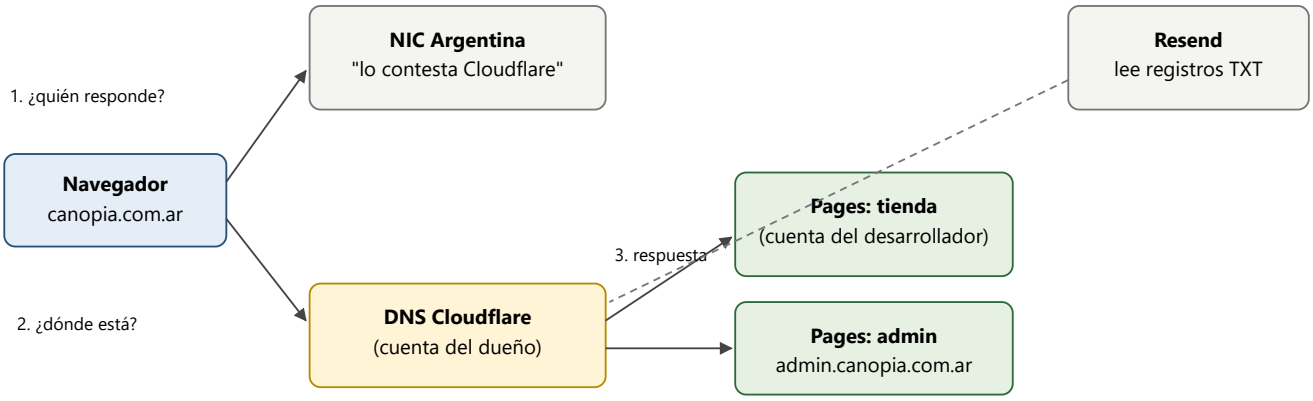


Figura 4. Cómo se resuelve el dominio: NIC.ar delega en el DNS de Cloudflare del dueño, que apunta a los proyectos de Pages del desarrollador.

Detalle técnico: en la práctica los navegadores no preguntan directo a NIC.ar sino a un "resolvedor" (el de su proveedor de Internet, por ejemplo), que hace el recorrido y guarda la respuesta un tiempo. Por eso los cambios de DNS pueden tardar de minutos a algunas horas en verse en todos lados.

9. Explicación técnica: cuentas y titularidad durante las cuotas

Este es el modelo acordado: el dueño es titular de su dominio, de su DNS y de sus datos desde el primer día; el desarrollador opera el hosting hasta que se complete el pago.

Recurso	Durante las cuotas	Después del pago total	En caso de mora
Dominio (NIC.ar)	Del dueño	Del dueño	Sigue siendo suyo
Cuenta Cloudflare (DNS + R2)	Del dueño; el desarrollador es miembro invitado	Del dueño; el desarrollador sale o queda si hay servicio técnico	Sigue siendo suya
Supabase (base, fotos, login)	Del dueño; desarrollador invitado	Ídem	Sigue siendo suya, con todos sus datos
Hosting (proyectos Pages)	**Del desarrollador**	Se recrean en la cuenta del dueño	**Se dan de baja**: el dominio queda sin web
Código fuente (GitHub)	Repositorio privado del desarrollador	Se entrega bajo licencia (copia en la cuenta del dueño)	No se entrega
Backups (workflow)	En un repositorio privado del desarrollador	Se mueve a la cuenta del dueño	A definir en el contrato (ver 17)
Resend (emails)	Cuenta del dueño, dominio verificado	Ídem	Sigue siendo suya

9.1 Alta de un cliente, paso a paso

1. El dueño compra el dominio en NIC.ar a su nombre (necesita su clave fiscal).
2. El dueño crea su cuenta gratuita de Cloudflare con su email y agrega el dominio. Cloudflare le da dos "nameservers".
3. En NIC.ar, el dueño cambia la delegación a esos nameservers (un formulario; se puede guiar por videollamada).
4. El dueño invita al desarrollador como miembro de su cuenta de Cloudflare y de su organización de Supabase.
5. El desarrollador crea los dos proyectos de Pages en su cuenta y los conecta al dominio: Pages puede usar un dominio que está en otra cuenta mediante registros CNAME, que el desarrollador carga en el DNS del dueño con su acceso de miembro.
6. El desarrollador configura Resend con el dominio del dueño (registros TXT/MX) y el bucket de R2 para backups.

9.2 Fin de las cuotas

Se crean los proyectos de Pages en la cuenta del dueño (o se le crea una cuenta nueva a su nombre), se conectan a su repositorio, se cambian los CNAME y se dan de baja los proyectos viejos. Es trabajo de minutos y sin corte perceptible.

10. Explicación técnica: Supabase free, tráfico e imágenes

10.1 Los límites del plan gratuito que importan

Recurso	Límite free (oct. 2026)	Uso esperado en un growshop
Base de datos	500 MB por proyecto	Con 100 reservas diarias crece unos 70 MB por año: años de margen
Storage (archivos)	1 GB	300 productos × 2 tamaños ≈ 15–25 MB
Egress (datos que salen)	5 GB sin caché + 5 GB con caché por mes (la documentación es inconsistente)	**El único límite que se puede romper**
Usuarios activos	50.000 por mes	Sobra
Pausa por inactividad	7 días sin tráfico	Con visitas reales no pasa; el keep-alive cubre la demo
Backups	No incluidos	Se resuelve con 5.6

Egress es todo lo que Supabase entrega hacia afuera: las respuestas de la base (listas de productos, pedidos) y, sobre todo, las imágenes. Las imágenes que salen desde la caché de la CDN de Supabase se cuentan aparte de las que salen "desde el origen".

10.2 Lo que se midió en la tienda real

Con la tienda local apuntando a la base de prueba: **62 imágenes WebP, promedio 19 KB, máximo 72 KB, 1,16 MB en total** para el catálogo completo. Servidas con `Cache-Control: max-age=3600` (el navegador las guarda una hora). Sin `loading="lazy"`: abrir `/productos` descarga todas.

10.3 Escenarios

Supuesto: 100 visitas diarias, 30 días. "Por visita" incluye catálogo, inicio, un par de fichas y las respuestas de la base.

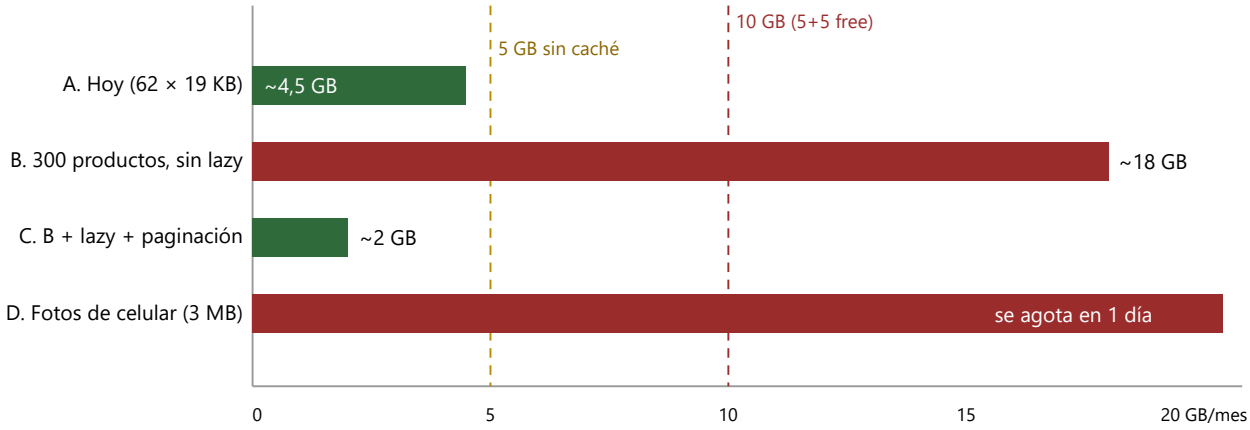


Figura 5. Egress mensual estimado por escenario, peor caso sin caché. El escenario D no entra en la escala: 62 fotos × 3 MB = 186 MB por visita al catálogo.

Conclusión

Lo que puede romper el plan free no es la cantidad de visitas sino el peso de las imágenes y cuántas se descargan por visita. Con las técnicas de 10.4, una tienda con 100–300 visitas diarias entra con margen. Alrededor de 500 visitas diarias conviene vigilar el panel de uso y considerar Supabase Pro.

10.4 Las técnicas para cuidar el plan free (todas decididas)

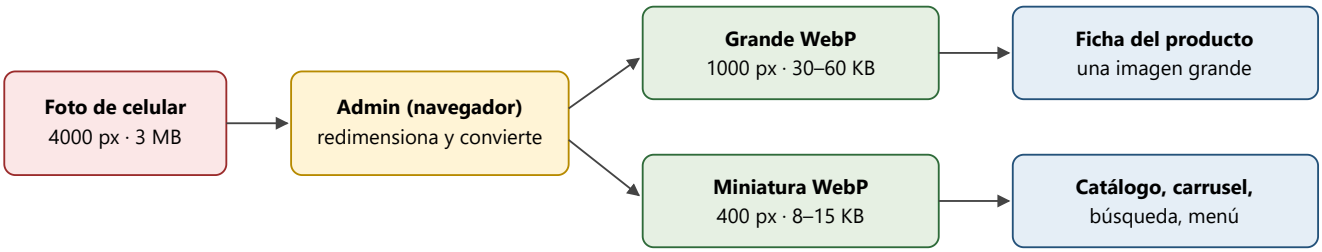


Figura 6. Pipeline de imágenes decidido: el admin genera dos versiones antes de subir; la tienda usa cada una donde corresponde.

#	Técnica	Qué hace	Ahorro
1	Compresión automática en el admin	Antes de subir, el navegador del dueño redimensiona y convierte a WebP con un `canvas`. Nadie tiene que acordarse de comprimir.	Evita el escenario D (×100)
2	Dos tamaños	Miniatura de 400 px para tarjetas y listas; grande de 1000 px solo en la ficha.	60–70 % del catálogo
3	Caché larga	Subir con `cacheControl` de un año. Es seguro porque cada archivo tiene nombre único (UUID) y nunca se sobrescribe: si cambia la foto, cambia el nombre.	Visitas repetidas ≈ 0
4	Carga diferida	`loading="lazy"` y `decoding="async"`: solo se descargan las imágenes que entran en pantalla. Declarar ancho y alto evita que la página "salte".	Proporcional a lo no visto
5	Paginación	Mostrar unos 24 productos por vez ("ver más" o páginas).	Grande con catálogos grandes
6	Columnas justas	Hoy el catálogo pide `select('*')`, que trae descripciones y especificaciones que la tarjeta no muestra.	Respuestas de la base más chicas
7	No repetir consultas	Cada componente que usa `usePaneles` o `useConfiguracionTienda` hace su propio pedido; compartirlos en la sesión.	Menos requests
8	Borrar huérfanas	Al reemplazar una foto o borrar un producto, borrar sus archivos del bucket.	Cuida el 1 GB
9	Logo liviano	De 720 KB a unos 30 KB (no consume Supabase, pero acelera la carga).	Velocidad
10	Monitoreo	Revisar mensualmente el panel de uso de Supabase o automatizar un aviso.	Detectar a tiempo
11	A futuro: fotos en R2	R2 no cobra el tráfico de salida. Si un cliente crece mucho, mover las fotos ahí deja a Supabase solo con datos.	Egress de fotos = 0

Las transformaciones de imágenes de Supabase (redimensionar al vuelo) son de planes pagos: por eso la compresión se hace en el navegador del admin.

10.5 Cómo funciona la caché en capas

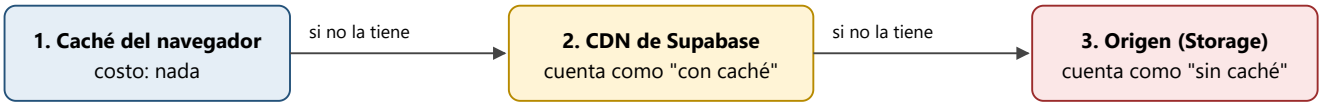


Figura 7. Cada imagen se busca primero en la capa más barata. La caché larga (técnica 3) hace que casi todo se resuelva en la capa 1 o 2.

11. Explicación técnica: backups

11.1 Por qué hacen falta

Supabase free no hace copias automáticas. Si alguien borra datos por error, si se rompe una migración o si la cuenta tiene un problema, sin backup se pierden pedidos, clientes y catálogo. Además, el backup de la base no incluye los archivos de Storage (las fotos): hay que copiarlos aparte.

11.2 La regla 3-2-1

Práctica estándar de la industria: **3** copias de los datos, en **2** soportes o proveedores distintos, con al menos **1** fuera del sistema principal. En Grow: la base viva en Supabase + la copia en R2 + la copia en Google Drive.

11.3 El circuito

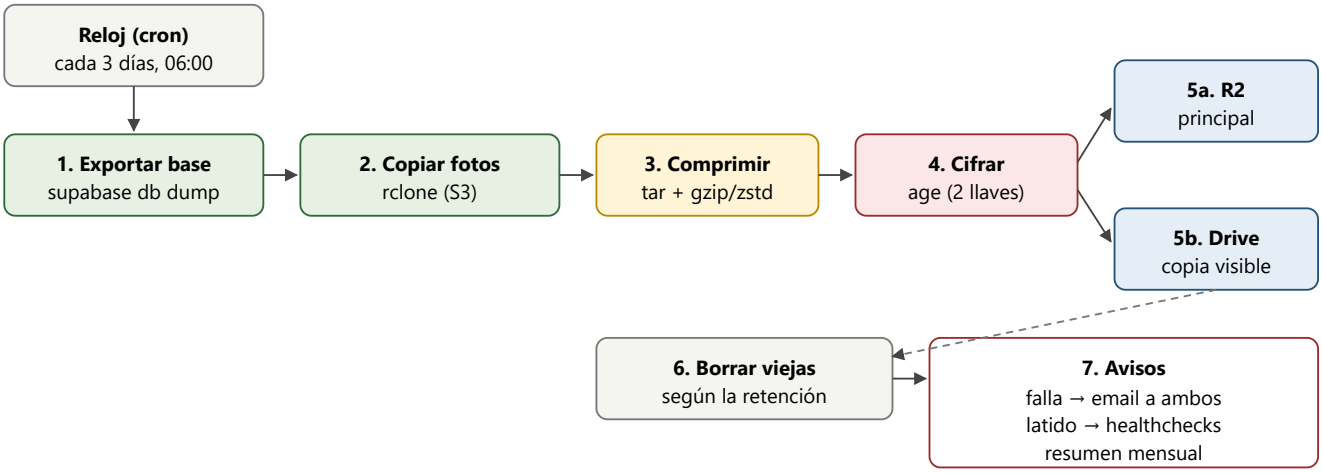


Figura 8. El backup completo, de punta a punta, en GitHub Actions.

1. **Exportar la base.** La guía oficial de Supabase usa `supabase db dump` en tres partes: roles, estructura y datos. Se conecta con la cadena de conexión de la base, guardada como *secret* del repositorio (nunca en el código).
2. **Copiar las fotos.** Storage tiene una API compatible con S3; `rclone` la lee (con la opción `--s3-list-version 2`, por un detalle conocido). Para no duplicar fotos en cada copia: espejo incremental (solo lo nuevo) y una foto completa del bucket una vez por mes.
3. **Comprimir** en un único archivo.
4. **Cifrar** con age para dos destinatarios (sección 12).
5. **Subir** a R2 y a Drive.
6. **Borrar** las copias que superan la retención.
7. **Avisar.** Tres mecanismos distintos, porque cubren fallas distintas (11.6).

11.4 R2 y Google Drive: diferencias

	Cloudflare R2	Google Drive del dueño
Pensado para	Programas (almacenamiento de objetos, API S3)	Personas

Gratis	10 GB-mes, sin costo de descarga	15 GB, compartidos con Gmail y Fotos
Acceso automático	Una clave de API limitada a un bucket, sin vencimiento: muy robusto	Delicado: en cuentas personales no sirven las "cuentas de servicio" (no tienen espacio propio). Hay que autorizar con el login del dueño y guardar ese permiso; si la app de Google queda en modo "prueba", el permiso vence en pocos días y el backup falla en silencio
Borrado de copias viejas	Reglas automáticas por antigüedad	Por script
Visibilidad para el dueño	Solo desde el panel de Cloudflare	Ve las copias en su Drive: tangible
Rol en Grow	**Principal**	**Secundario y visible**

Usar ambos da redundancia real: si falla la autorización de Drive, R2 sigue funcionando, y si algo pasara con la cuenta de Cloudflare, el dueño tiene sus copias en Google.

11.5 Frecuencia y retención

Decidido: **cada 3 días**. Propuesta de retención para confirmar cuando el código esté terminado:

- **Todas las copias de los últimos 90 días** (unas 30 copias).
- **La primera copia de cada mes durante 12 meses** (12 copias más), para poder volver a un estado viejo si un error se descubre tarde.

Tamaño estimado: la base de un growshop pesa pocos MB comprimida, incluso con años de pedidos. Unas 42 copias de la base caben holgadas en los 10 GB de R2 y en el Drive.

> **Detalle técnico del "cada 3 días"**
> La expresión cron `0 6 */3 * *` corre los días 1, 4, 7… 28 y 31 de cada mes, y vuelve a empezar el día 1: a fin de mes puede haber dos copias seguidas (31 y 1) o un hueco de 4 días. Alternativa exacta: que el workflow corra todos los días y salga sin hacer nada salvo cada tercer día contado desde una fecha fija. Es un detalle de implementación.

11.6 Avisos

Mecanismo	Qué detecta	A quién
Email en caso de error	El workflow corrió y falló (la base no respondió, se venció el permiso de Drive, etc.)	Desarrollador y dueño
"Latido" (healthchecks.io u otro gratuito)	El workflow **no corrió**: el servicio espera un ping cada 3 días y avisa si no llega. Es la falla que nadie más ve.	Desarrollador y dueño
Resumen mensual	Tranquilidad: "10 copias correctas este mes, última el día 28, 3,2 MB".	Dueño (y desarrollador)

11.7 Probar la restauración

Un backup que nunca se restauró no está probado. Antes de vender: restaurar una copia en un proyecto de Supabase de prueba (descifrar con la llave, cargar roles, estructura y datos, re-subir fotos) y documentar los pasos. Repetirlo una vez por año.

12. Explicación técnica: cifrado con age y la "doble llave"

12.1 Tres capas de cifrado distintas

Capa	Qué protege	Quién tiene las claves	Estado
En tránsito (HTTPS/TLS)	Los datos mientras viajan entre el navegador y los servidores	Automático	Ya existe
En reposo en Supabase (AES-256)	Los discos y backups de Supabase; claves por proyecto en módulos de seguridad de hardware	Supabase	Ya existe
Copias de seguridad (age)	Los archivos de backup en R2 y Drive	**Solo el dueño y el desarrollador**	A implementar

> **Qué se puede prometer y qué no**
> Se puede decir con verdad: "Tus datos viajan cifrados, se guardan cifrados y cada 3 días se hace una copia de seguridad cifrada con una llave que solo tenés vos". No conviene decir "indescifrables" a secas: la aplicación necesita leer los datos para funcionar, y quien tenga la contraseña del admin los ve.

12.2 Cifrado simétrico y asimétrico

Simétrico: la misma clave cierra y abre, como un candado con una sola llave. Problema: el proceso automático que cifra necesitaría esa clave, y quien la robe de GitHub podría abrir todas las copias.

Asimétrico (de clave pública): cada persona tiene un par de claves. La **clave pública** funciona como un candado abierto que cualquiera puede cerrar; la **clave privada** es la única llave que lo abre. Se puede repartir la pública sin riesgo: solo sirve para cerrar.

age es una herramienta moderna, simple y auditada, que combina ambos mundos.

12.3 Cómo funciona age por dentro

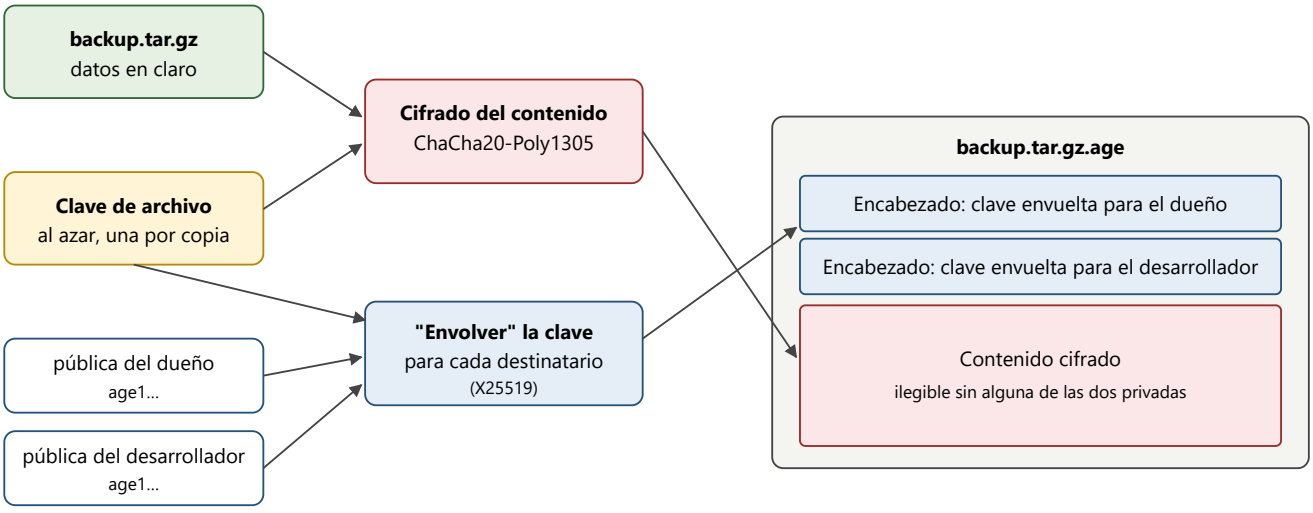


Figura 9. age cifra el contenido con una clave aleatoria y guarda esa clave "envuelta" una vez por cada destinatario.

1. Para cada copia, age genera una **clave de archivo** aleatoria y cifra el contenido con ella (algoritmo ChaCha20-Poly1305, que además detecta cualquier alteración).
2. Esa clave de archivo se "envuelve" (se cifra) con la clave pública de **cada destinatario** (algoritmo X25519) y se guarda en el encabezado del archivo.
3. Para abrir, cualquiera de los destinatarios usa su clave privada para desenvolver la clave de archivo y, con ella, descifra el contenido.

Las claves son texto corto: la pública empieza con `age1` (unos 62 caracteres) y la privada con `AGE-SECRET-KEY-1` (unos 74). La pública se guarda como configuración del workflow; la privada nunca sale de manos de su dueño.

12.4 La "doble llave" decidida

Lo que pediste ("doble validación, yo tengo una copia de la llave") se resuelve mejor de lo que parece: **no hace falta compartir una llave**. Cada uno tiene su propio par de claves y el backup se cifra para los dos. El dueño abre con la suya, el desarrollador con la suya. Nadie conoce la llave del otro.

12.5 Qué pasa al terminar las cuotas (rotación)

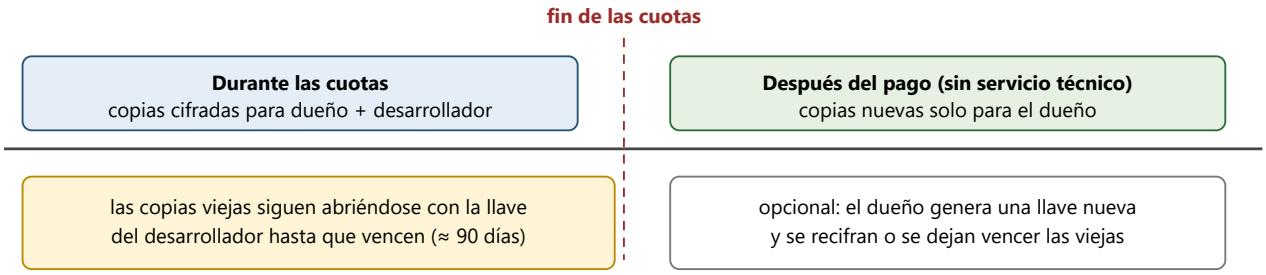


Figura 10. Rotación de destinatarios al terminar las cuotas.

- **Sacar al desarrollador:** basta con quitar su clave pública de la lista de destinatarios. Las copias nuevas solo las abre el dueño. **No afecta los datos**, solo quién puede abrir las copias.
- **Las copias anteriores** siguen abriéndose con la llave del desarrollador, porque se cifraron cuando él era destinatario. Con una retención de 90 días se van renovando solas; si se quiere un corte inmediato, se pueden descifrar y volver a cifrar solo para el dueño (un script de minutos). Las copias mensuales de 12 meses tardan más en vencer: tenerlo en cuenta en el contrato.
- **Llave nueva del dueño:** es opcional y sigue la misma lógica: se agrega la nueva pública, se quita la vieja y se recifra o se deja vencer.
- **Contrato de servicio técnico:** se vuelve a agregar la clave pública del desarrollador. Mientras dure, puede restaurar; al terminar, se quita. Nunca hace falta que el dueño entregue su llave privada.

12.6 Dónde guarda el dueño su llave privada

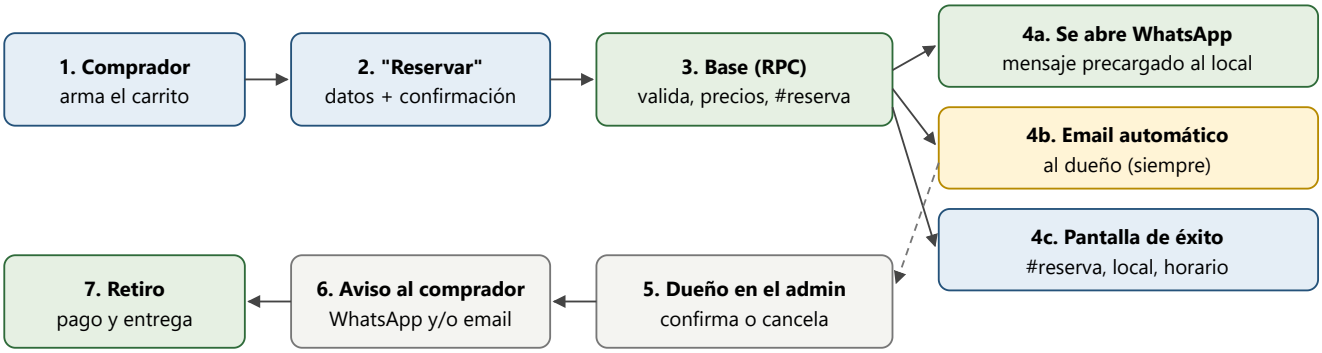
- **Un pendrive común alcanza.** No hace falta un dispositivo especial: la llave es una línea de texto.
- **Protegida con contraseña:** age permite guardar la llave privada cifrada con una frase de paso. Si el pendrive se pierde, quien lo encuentre no puede usarla.
- **Dos copias:** el pendrive y una segunda copia en otro lugar (un gestor de contraseñas como Bitwarden, o la llave impresa en papel dentro de un sobre en un lugar seguro).
- **Revisión anual:** los pendrives se degradan; una vez por año, comprobar que abre (se puede hacer como parte de la prueba de restauración, 11.7).

- **Si el dueño la pierde y no hay otra:** con la doble llave, el desarrollador todavía puede abrir. Esa es una ventaja concreta del esquema decidido.

> **Valor comercial**
> "Tus datos son tuyos desde el primer día: copias cifradas cada 3 días, en dos nubes distintas, con una llave que guardás vos; te avisamos si algo falla y te mandamos un resumen todos los meses." Pocos sistemas a medida para comercios chicos ofrecen esto.

13. Explicación técnica: el flujo de reserva y los avisos

13.1 De punta a punta



El email del paso 4b lo dispara la base al insertarse la reserva (trigger o Database Webhook → Edge Function → Resend), así sale aunque el comprador cierre la página o no tenga WhatsApp. El WhatsApp del paso 4a depende de que el comprador lo envíe.

El contrato de compraventa se perfecciona en el paso 7 (en el local), no en el paso 2.

Figura 11. Flujo de una reserva con los avisos decididos.

13.2 Estados de la reserva

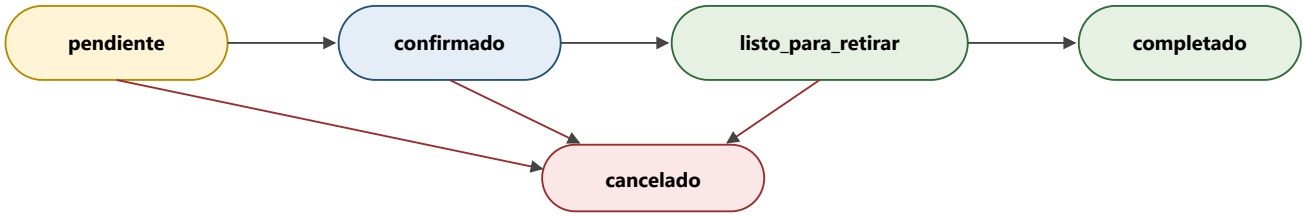


Figura 12. Transiciones permitidas, impuestas por el trigger `trg_validar_transicion_pedido`. Completado y cancelado son finales.

Los nombres internos en la base (`pedidos`, `pendiente`, etc.) se mantienen: cambiar nombres de tablas o estados en producción es riesgoso y no aporta. Lo que cambia es el **lenguaje visible** (ver 17 sobre las etiquetas exactas).

13.3 Las opciones de WhatsApp que se evaluaron

Opción	Cómo funciona	Costo	Contras
Mensaje desde el comprador (elegida)	Se abre `wa.me/<número del local>?text=…` con el mensaje armado; el comprador lo envía.	$0	Depende de que el comprador lo envíe; por eso el email al dueño

API oficial (WhatsApp Business Platform)	Un número registrado en Meta envía plantillas aprobadas al dueño. Existe la "coexistencia" para usar la API con el número del local sin perder la app.	Por mensaje (desde el 1/10/2026 también los de servicio y utilidad)	Verificación del negocio, plantillas, alta compleja; si el número es del desarrollador, la dependencia sobrevive a la entrega
Librerías no oficiales	Simulan WhatsApp Web en un servidor.	Servidor siempre prendido	Violan los términos de Meta; riesgo de bloqueo del número

14. Explicación técnica: pasarela de pago (para el futuro)

Decidido no implementarla ahora (5.4). Queda el análisis completo en `docs/auditoria/pasarela-de-pago.md`; acá, lo conceptual para estudiar.

14.1 Cómo funciona un cobro con Mercado Pago Checkout Pro

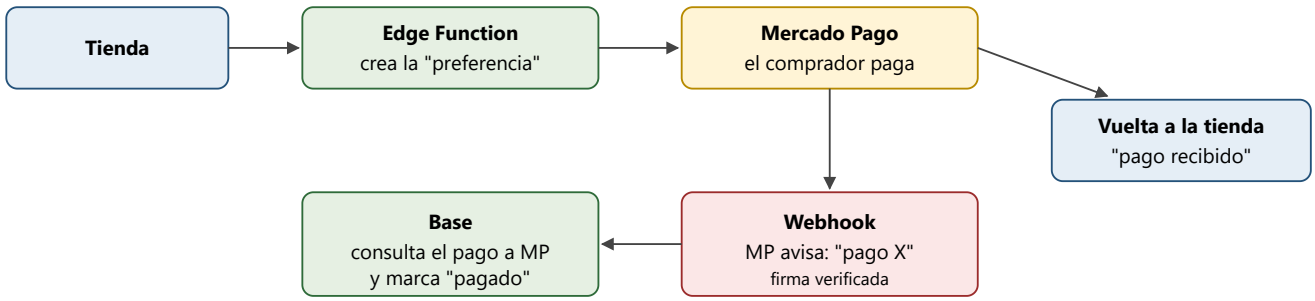


Figura 13. El navegador nunca decide si se pagó: lo confirma el webhook, verificado y consultado contra Mercado Pago.

14.2 Conceptos clave

- **Webhook:** un aviso que un servicio externo le manda a tu sistema cuando pasa algo ("el pago 123 fue aprobado"). Es una llamada HTTP a una URL tuya.
- **Firma (HMAC):** como cualquiera podría llamar a esa URL, Mercado Pago firma cada aviso con un secreto compartido; tu función recalcula la firma y descarta lo que no coincide.
- **Idempotencia:** Mercado Pago reintenta los avisos si no respondés a tiempo (22 segundos). El sistema tiene que tolerar recibir el mismo aviso varias veces sin acreditar dos veces.
- **No confiar en el body:** ante cada aviso se consulta el pago a la API de Mercado Pago y se compara monto, moneda y referencia.

14.3 Datos de referencia (consultados el 23/10/2026)

Plazo de acreditación	Comisión Checkout (Mercado Pago Argentina)
Al instante	6,29 % + IVA
10 días	4,39 % + IVA
18 días	3,39 % + IVA
35 días	1,49 % + IVA

Variables por impuestos provinciales. **Riesgo de rubro:** según notas de prensa, los términos de Mercado Pago prohíben la parafernalia; antes de integrar, cada dueño debería confirmar por escrito que su catálogo está aceptado. Alternativa sin comisión: transferencia con comprobante y verificación manual.

15. Orientación legal

> **Aclaración**
> Esta sección es orientación general basada en normativa pública consultada el 23/10/2026. No es asesoramiento legal. El dueño del proyecto es estudiante de abogacía y redactará los textos; esto sirve como punto de partida y lista de verificación.

15.1 Botón de arrepentimiento

La norma: la Resolución SCI 424/2020 obliga a quienes comercializan bienes y servicios a través de sitios web a publicar un enlace "Botón de arrepentimiento" de acceso fácil y directo desde la página de inicio, sin exigir registro previo, y a informar un código de revocación dentro de las 24 horas. El derecho de fondo es el de revocación del art. 34 de la Ley 24.240 (10 días corridos desde la entrega o la celebración, lo último) y el art. 1110 del CCyC.

La postura del proyecto: no hay venta a distancia sino una reserva; el contrato se perfecciona en el local, con el producto a la vista, el pago y la entrega. Así desaparece la razón de ser del arrepentimiento (no haber visto el bien), y en la práctica el comprador puede simplemente no retirar.

Condiciones para que la postura se sostenga:

- Coherencia total del lenguaje: "reservar", "solicitud de reserva", "reserva recibida". Nada de "comprar", "compra" ni "Compra Protegida".
- Los Términos lo dicen expresamente: la reserva no obliga al pago, y la compraventa se celebra en el local.
- Si en el futuro se agrega cobro online, la postura cae y el botón pasa a ser obligatorio.

Contrapunto: la oferta pública obliga a quien la emite (art. 7 de la Ley 24.240), y podría sostenerse que la reserva aceptada ya forma un contrato. El riesgo se considera bajo, y el botón es barato de implementar si alguna vez se quiere eliminar la discusión.

15.2 La oferta: stock y error de precio

El art. 7 de la Ley 24.240 dice que la oferta dirigida a consumidores potenciales indeterminados obliga a quien la emite durante el tiempo en que se realice, debiendo contener fecha de comienzo y finalización, y sus **modalidades, condiciones o limitaciones**. De ahí se desprende lo que los Términos deben hacer:

- **No** invocar el art. 7 como fuente de una facultad de corregir precios: el artículo obliga al oferente.
- Declarar las limitaciones de la oferta: sujeta a disponibilidad de stock; precios vigentes al momento de la reserva y por un plazo determinado (por ejemplo, hasta la fecha de retiro acordada).
- Tratar el error de precio como **error reconocible** (arts. 265–266 del CCyC): un precio manifiestamente equivocado no obliga si el error era reconocible para el destinatario.
- Evitar cláusulas amplias del tipo "nos reservamos el derecho de modificar precios en cualquier momento", que pueden considerarse abusivas (art. 37 de la Ley 24.240).

15.3 Datos del proveedor

La Resolución 270/2020 incorporó la Res. GMC 37/2019 del Mercosur sobre comercio electrónico: exige informar nombre comercial y razón social, CUIT, domicilio físico y electrónico y medio de atención al consumidor. Decidido: no cargarlos en la demo (no hay titular real), dejar los campos preparados en Configuración y

completarlos al personalizar para cada cliente. Guías del sector mencionan además el QR de "Data Fiscal" (Formulario 960/D de ARCA): verificar con el contador de cada cliente.

15.4 Datos personales (Ley 25.326)

- **Información al recolectar** (según la AAIP): finalidad, destinatarios, identidad y domicilio del responsable, carácter obligatorio o facultativo de los datos, y los derechos de acceso, rectificación y supresión, en un lugar visible.
- **Política de privacidad** genérica, completada al personalizar: qué datos (nombre, teléfono, email, historial de reservas), para qué (gestionar reservas y contactar al cliente), dónde se alojan (Supabase, Cloudflare, Resend, Google: proveedores fuera del país), cuánto tiempo, cómo ejercer los derechos.
- **Inscripción de la base:** la AAIP indica que se inscriben las bases públicas y las privadas destinadas a dar informes; si una base de clientes propia entra, conviene confirmarlo. Se tramita por TAD.
- **Derecho de supresión:** hoy no hay forma de borrar una cuenta (pendiente en el plan).
- **Exportación a Excel:** el archivo sale del sistema; mencionarlo como uso interno.

15.5 Rubro y edad

- El art. 12 de la Ley 23.737 castiga a quien preconizare o difundiere públicamente el uso de estupefacientes. El contenido del catálogo, las descripciones y las reseñas de ejemplo deben ser neutros (cultivo, jardinería, iluminación, accesorios).
- Productos de tabaco (papelillos, filtros) tienen restricción de venta a menores. Queda abierto si se agrega una confirmación "+18" (sección 17).

15.6 Software, licencias y DNDA

La Ley 11.723 protege el software como obra. La protección nace con la creación; el registro sirve como prueba de autoría y fecha. La **Dirección Nacional del Derecho de Autor (DNDA)**, del Ministerio de Justicia, ofrece por TAD:

Trámite	Para qué	Arancel informado
Depósito en custodia de obra inédita de software	Código no publicado. Protege la prueba por 3 años, renovables dentro de los 30 días posteriores al vencimiento; si no se renueva, la obra se destruye.	$1.400
Registro de software publicado	La obra debe estar publicada antes del trámite.	$3.800
Inscripción de contratos sobre software	Registrar, por ejemplo, contratos de licencia.	Ver trámite

Sugerencia: depositar el código base como obra inédita (lo que ve el navegador es el resultado compilado; el código fuente no se publica). Montos a confirmar en argentina.gob.ar antes de pagar.

Licencia perpetua y no exclusiva: el cliente puede usar, modificar y alojar su copia para siempre; el desarrollador conserva la autoría y puede licenciar el mismo código base a otros growshops; lo que se haga con otros clientes no afecta la copia entregada. La personalización puede quedar dentro de la licencia o cederse por separado.

15.7 Condiciones de los proveedores (una lección de esta auditoría)

El plan Hobby de Vercel prohíbe el uso comercial. Antes de elegir cualquier servicio gratuito para un cliente, revisar sus condiciones de uso comercial y de rubro. Resumen de lo verificado: Vercel Hobby no; Cloudflare Pages free sí (según su personal); Netlify free sí, con tope de créditos; Mercado Pago con restricciones de rubro.

16. Plan de trabajo paso a paso

Para la próxima etapa (sesión de código). Cada tarea indica objetivo, dónde se trabaja y cuándo está terminada. El orden respeta dependencias. Las reglas del proyecto aplican siempre: `npm run typecheck` y build de la app tocada; verificación en el navegador (1366, ~1920, ~1000 y 375 px) para cambios visuales; base solo con cambios incrementales y regeneración de tipos.

Fase 1 — Antes de ofrecer el producto

1.1 Reseñas de ejemplo genéricas S
Objetivo: 10 reseñas con nombres ficticios y textos genéricos sin referencias a ninguna tienda; avatares SVG ilustrados variados; quitar "4,8 (232)".
Dónde: `client/src/components/home/ReviewsCarousel/`, borrar `client/public/1.jpg…10.jpg`.
Terminado cuando: no queda ninguna foto ni dato de personas o tiendas reales; el carrusel funciona igual.

1.2 "Datos protegidos" S
Objetivo: quitar "Compra Protegida" de catálogo y ficha; en la reserva, sello "Datos protegidos" con link a la política de privacidad.
Dónde: `ProductsPage.tsx:163`, `ProductDetailPage.tsx:227`, `ResumePage.tsx:375` y sus CSS.

1.3 Lenguaje de "reserva" S–M
Objetivo: textos visibles de tienda y admin coherentes con el modelo de reserva (botones, títulos, confirmación, emails, mensajes de WhatsApp). Sin renombrar tablas ni estados en la base.
Depende de: la decisión de etiquetas (17).

1.4 Variantes sin borrado + snapshot del nombre M
Objetivo: editar un producto actualiza, crea o borra solo las variantes que cambiaron (por id), en una operación transaccional (RPC). Agregar `nombre_variante` a `pedido_items` (columna nueva + backfill + RPC `crear_pedido_completo` + tipos y mappers) y usarla en admin, Mi Cuenta y emails.
Dónde: `admin/src/components/ProductForm.tsx:115-157`, `shared/rpc.sql`, `shared/schema.sql`, Edge Function.
Terminado cuando: editar un producto no cambia los ids de sus variantes y un pedido viejo sigue mostrando su variante.

1.5 Teléfonos normalizados M
Objetivo: validar y guardar en formato `549…` los teléfonos del comprador, del registro y del local; quitar el `54` fijo del admin; migrar los existentes.
Dónde: `ResumePage.tsx`, `RegisterPage.tsx`, `SettingsPage.tsx`, `OrdersPage.tsx:306,522`, `CustomersPage.tsx:144`, `shared/rpc.sql:56`, links `wa.me` del client.
Terminado cuando: los links de WhatsApp abren el chat correcto con números reales de prueba.

1.6 Reserva con WhatsApp del comprador M
Objetivo: al confirmar, abrir `wa.me/<teléfono del local>?text=…` con número de reserva, items, total y nombre; pantalla de éxito con número, local y horario, y botón para reintentar el WhatsApp.
Depende de: 1.5.

1.7 Email automático al dueño M
Objetivo: campo "email de avisos" en Configuración; al insertarse una reserva, la base dispara una Edge Function (trigger con `pg_net` o Database Webhook) que envía el email vía Resend. La función se autentica con un secreto propio, no con JWT.
Depende de: dominio verificado en Resend por cliente (en la demo, la casilla del desarrollador).

1.8 Pipeline de imágenes M
Objetivo: compresión en el admin a WebP (grande 1000 px, 30–60 KB; miniatura 400 px, 8–15 KB); guardar ambas URL (columna nueva o convención de nombres); `cacheControl` de un año al subir; borrar archivos al reemplazar o eliminar.
Dónde: `ProductForm.tsx`, `shared/src/constants` (usar la constante existente), tipos y mappers.

1.9 Tienda liviana M
Objetivo: miniaturas en tarjetas, carruseles, búsqueda y menú; `loading="lazy"`, `decoding="async"`, ancho y alto; paginación del catálogo; columnas justas en las consultas; datos compartidos entre componentes; logo optimizado.
Terminado cuando: abrir `/productos` descarga solo las imágenes visibles.

1.10 Términos y política de privacidad S–M
Objetivo: textos redactados por el desarrollador según 15.1–15.4; campos de datos del proveedor preparados y vacíos en Configuración.

1.11 Migración a Cloudflare Pages S
Objetivo: archivos `_headers` y `_redirects` equivalentes a los `vercel.json`; dos proyectos con *build watch paths*; variables de entorno; probar CSP con la base real.

1.12 Backups M
Objetivo: workflow cada 3 días (11.3), cifrado age con dos destinatarios, R2 + Drive, retención (11.5), avisos (11.6), prueba de restauración documentada (11.7).

1.13 SQL al día S
Objetivo: llevar a `shared/*.sql` el CHECK de `clientes` y los límites del bucket que hoy solo están en `correcciones.sql`; keep-alive que consulte una tabla.

Fase 2 — Primeras ventas

#	Tarea	Nota
2.1	Carrito persistente en `localStorage`, re-precio al cargar, ficha en la misma pestaña; mostrar el total que devuelve la base	C1, C2
2.2	Sucursal de retiro, observaciones y franja de retiro en la reserva	Hoy se coordina a mano
2.3	Motivo de cancelación y aviso al comprador	P2
2.4	Recordatorio y vencimiento de reservas no retiradas	P3, pg_cron
2.5	Recuperar contraseña; errores de login en español	C6
2.6	Baja de cuenta y anonimización	Ley 25.326

2.7	SEO básico: meta y Open Graph por producto (función en Pages), `robots.txt`, `sitemap.xml`	Para que los links compartidos muestren foto
2.8	Reportes: ventas por período, ticket promedio, más vendidos	Resuelve también P6 en el Dashboard
2.9	Búsqueda de reservas por teléfono, email y número	P4
2.10	Horarios por día y por sucursal	C9
2.11	Analytics sin cookies (ajustar CSP y política)	
2.12	Edición masiva de precios por porcentaje	Contexto inflacionario
2.13	Interruptor en el admin para ocultar la sección de reseñas	Según 17

Fase 3 — Escala

#	Tarea
3.1	Transferencia con comprobante y, si el rubro lo permite, Mercado Pago (`pasarela-de-pago.md`)
3.2	Stock numérico opcional, descontado al confirmar
3.3	Varias imágenes por producto
3.4	Estado "sin stock" para variantes
3.5	Paginación y agregados en SQL para Pedidos y Clientes (más de 1000 filas)
3.6	Reseñas reales administrables o enlace a Google
3.7	Clave de idempotencia en la reserva
3.8	Fotos en R2 si un cliente crece mucho
3.9	Modo simple de paneles ("solo categorías")
3.10	Etapa mobile del admin

17. Preguntas abiertas

#	Pregunta	Recomendación
1	¿Qué etiquetas exactas usa el admin para el lenguaje de reserva?	Sección "Reservas" en el menú; estados visibles: Nueva, Confirmada, Lista para retirar, Retirada, Cancelada. Mismo vocabulario que ve el comprador en WhatsApp y emails, así el dueño y el cliente hablan igual.
2	¿Teléfono en uno o dos campos (área + número)?	Dos campos: reduce errores de carga del 0 y el 15.
3	Retención de backups	90 días completos + 12 mensuales (11.5), a confirmar con el código terminado.
4	¿Qué pasa con los backups y su repositorio si el cliente entra en mora?	Que el contrato lo prevea: los datos son del dueño; el workflow podría seguir corriendo o pasar a su cuenta.
5	¿Confirmación "+18" en la reserva?	Sí, una casilla simple; el control real es al retirar.
6	¿La sección de reseñas se puede apagar desde el admin?	Sí (tarea 2.13), útil al entregar a un cliente sin reseñas propias todavía.
7	¿Los avisos de backup van por email o también por WhatsApp/Telegram?	Email al inicio (gratis y simple).
8	¿Se deposita el código base en la DNDA y cuándo?	Al cerrar la fase 1, como obra inédita.

18. Glosario

Término	Significado
age	Herramienta de cifrado de archivos con claves públicas y privadas.
Anon key	Clave pública de Supabase que va en el navegador; por sí sola no da permisos: los da RLS.
Backup	Copia de seguridad de los datos.
Bucket	"Carpeta" de almacenamiento de archivos (Supabase Storage, R2).
Build	Compilación del código fuente a los archivos finales.
Caché	Copia guardada cerca de quien la usa para no volver a descargarla.
CDN	Red de servidores repartidos que entregan archivos desde el más cercano.
CNAME	Registro DNS que hace que un nombre sea alias de otro.
Cron	Forma de programar tareas periódicas ("cada 3 días a las 6").
CSP	Política de seguridad de contenido: header que limita de dónde puede cargar cosas la página.
DNS	Sistema que traduce nombres de dominio a direcciones.
Edge Function	Función que corre en los servidores de Supabase (por ejemplo, para mandar emails con claves secretas).
Egress	Datos que salen de un servicio hacia los usuarios; en Supabase free es el límite que más importa.
GitHub Actions	Servicio de GitHub para correr tareas automáticas (builds, backups).
HMAC	Firma criptográfica con un secreto compartido para comprobar que un mensaje es auténtico.
Idempotencia	Propiedad de una operación que, repetida, produce el mismo resultado que una sola vez.
Lazy loading	Carga diferida: descargar una imagen solo cuando va a verse.
Nameservers	Los servidores que contestan el DNS de un dominio; se configuran en NIC.ar.
R2	Almacenamiento de archivos de Cloudflare, sin costo por descarga.
RLS	Seguridad a nivel de fila: reglas en la base que deciden qué filas puede ver o tocar cada usuario.
RPC	Función de la base que se llama desde la app (por ejemplo, `crear_pedido_completo`).
SPA	Aplicación de una sola página: el navegador carga una vez y cambia de pantalla con JavaScript.
Webhook	Aviso HTTP que un servicio externo manda a tu sistema cuando ocurre algo.
WebP	Formato de imagen moderno, más liviano que JPG y PNG a igual calidad.

19. Fuentes consultadas

Todas consultadas el 23 de octubre de 2026. Los montos y condiciones pueden cambiar: verificar antes de usarlos.

Normativa argentina

- Resolución SCI 424/2020 (botón de arrepentimiento): https://www.argentina.gob.ar/normativa/nacional/norma-342869/texto
- Resolución 270/2020 (comercio electrónico, Res. GMC 37/19): https://www.argentina.gob.ar/normativa/nacional/norma-341933/texto · https://servicios.infoleg.gob.ar/infolegInternet/anexos/340000-344999/341933/norma.htm
- Ley 24.240 art. 34, comentario: https://www.marval.com/Publicacion/en-los-contratos-electronicos-el-consumidor-tiene-la-facultad-de-revocarlo-por-la-misma-via-12734
- DNU 274/2019 (lealtad comercial): https://www.consejosalta.org.ar/wp-content/uploads/DECRETO-274.pdf
- AAIP, obligaciones de responsables de bases: https://www.argentina.gob.ar/node/53779
- Ley 23.737: https://sherloc.unodc.org/cld/uploads/res/document/ley-23737-regimen-penal-de-estupefacientes_html/Ley_23737_regimen-penal-de-estupefacientes.pdf
- DNDA, depósito de software inédito: https://www.argentina.gob.ar/servicio/deposito-en-custodia-de-obra-inedita-software
- DNDA, registro de software publicado: https://www.argentina.gob.ar/servicio/registrar-un-software-puesto-en-conocimiento-publico
- DNDA, inscripción de contratos sobre software: https://www.argentina.gob.ar/servicio/inscribir-un-contrato-sobre-software-obra-multimedia-pagina-web-o-videojuego
- Data Fiscal (guía de terceros): https://ayuda.tiendanube.com/arca/como-agregar-el-codigo-de-data-fiscal-de-arca

Infraestructura

- Vercel, plan Hobby y precios: https://vercel.com/docs/plans/hobby · https://vercel.com/pricing · https://vercel.com/docs/limits/fair-use-guidelines
- Cloudflare Pages, límites: https://developers.cloudflare.com/pages/platform/limits/
- Cloudflare, uso comercial en el plan gratuito (foro): https://community.cloudflare.com/t/is-commercial-use-allowed-on-the-free-plan-for-pages-functions-kv/971058
- Cloudflare, Workers vs Pages: https://developers.cloudflare.com/workers/best-practices/workers-best-practices/ · https://developers.cloudflare.com/workers/static-assets/migration-guides/migrate-from-pages/
- Cloudflare R2, precios: https://developers.cloudflare.com/r2/pricing/
- Netlify, planes por créditos: https://docs.netlify.com/manage/accounts-and-billing/billing/billing-for-credit-based-plans/credit-based-pricing-plans/
- Supabase, backups: https://supabase.com/docs/guides/platform/backups · backups con GitHub Actions: https://supabase.com/docs/guides/deployment/ci/backups
- Supabase, egress: https://supabase.com/docs/guides/platform/manage-your-usage/egress · precios: https://supabase.com/pricing
- Supabase, copia de Storage por S3: https://supabase.com/docs/guides/self-hosting/copy-from-platform-s3
- Supabase, cifrado en reposo: https://supabase.com/legal/privacy-resources/data-residency-and-transfers-faq
- Supabase, Database Webhooks: https://supabase.com/docs/guides/database/webhooks
- age: https://github.com/FiloSottile/age · especificación: https://age-encryption.org/v1
- Google Drive y cuentas de servicio: https://fivetran.com/docs/activations/destinations/available-destinations/google-drive/troubleshooting/service-accounts-storage-quota

Mensajería y pagos

- WhatsApp Business Platform, precios: https://developers.facebook.com/documentation/business-messaging/whatsapp/pricing
- WhatsApp, coexistencia con la app Business: https://developers.facebook.com/documentation/business-messaging/whatsapp/embedded-signup/onboarding-business-app-users/
- Mercado Pago, costos de Checkout: https://www.mercadopago.com.ar/herramientas-para-vender/check-out
- Mercado Pago, notificaciones: https://www.mercadopago.com.ar/developers/es/docs/checkout-pro/payment-notifications

- Bienes prohibidos (nota de prensa): https://www.canal26.com/general/2025/07/12/mercado-pago-podra-cerrar-cuentas-que-realicen-compras-prohibidas-que-productos-estan-vetados/